

Clase 228 — Seguridad de Kubernetes: arquitectura

Parte: **10 — Seguridad en la nube y contenedores** · Fuente: *Martin & Hausenblas, "Hacking Kubernetes" (O'Reilly)* y *documentación oficial de Kubernetes* 🕒 Duración estimada: **120 min** · Nivel: **Intermedio**

Objetivo

Comprender la arquitectura de Kubernetes desde la perspectiva de la seguridad: los componentes del plano de control (API server, etcd, scheduler, controller manager) y del plano de datos (kubelet, kube-proxy, container runtime), cómo se comunican y dónde están sus superficies de ataque. Es la base para el hardening y los ataques de la clase siguiente.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Describir** los componentes del plano de control y de datos y su función.
2. **Identificar** las superficies de ataque de cada componente (API, etcd, kubelet).
3. **Explicar** el flujo de una petición: autenticación, autorización (RBAC) y admission control.
4. **Distintuir** los objetos de seguridad clave: ServiceAccount, Secret, NetworkPolicy, RBAC.
5. **Desplegar** un clúster de laboratorio para practicar.

Temas

#	Tema	Por qué importa
1	Plano de control vs plano de datos	Separa cerebro y músculo del clúster
2	API server como punto central	Todo pasa por él; es el objetivo principal
3	etcd	Almacena TODO el estado, incluidos Secrets
4	kubelet	Agente por nodo; su API es un vector clásico
5	Flujo authn → authz → admission	Cómo se autoriza cada acción
6	RBAC y ServiceAccounts	Identidad y permisos dentro del clúster
7	Namespaces y NetworkPolicy	Aislamiento lógico y de red

Definiciones y características

- **API server:** frontend REST del plano de control. *Clave:* toda operación pasa por authn, authz y admission aquí.
- **etcd:** base clave-valor con el estado del clúster. *Clave:* contiene los Secrets; su compromiso equivale a comprometer el clúster.

- **kubelet:** agente que ejecuta pods en cada nodo. *Clave:* su API (10250) sin autenticación permite ejecutar en pods.
- **RBAC:** control de acceso por roles (Role/ClusterRole + Binding). *Clave:* privilegio mínimo dentro del clúster.
- **ServiceAccount:** identidad de los pods frente al API server. *Clave:* su token, si se roba, da acceso a la API.
- **Admission controller:** valida/mutila objetos antes de persistirlos (p. ej. Pod Security Admission). *Clave:* fuerza políticas de seguridad.
- **NetworkPolicy:** firewall L3/L4 entre pods. *Clave:* sin ella, todos los pods se comunican libremente.

Herramientas y preparación

- **kind** o **minikube** para un clúster local de laboratorio.
- **kubect** configurado; **kube-bench** y **kubeaudit** (se profundizan en la clase 229).
- Opcional: **Lens** o **k9s** para visualizar el clúster.

```
# Crear un clúster de laboratorio con kind
kind create cluster --name lab
# Ver los componentes del plano de control
kubectl get pods -n kube-system
# Inspeccionar el flujo de autorización de una acción
kubectl auth can-i create pods --as system:serviceaccount:default:default
```

Laboratorio guiado

1. Crea un clúster con `kind create cluster` y examina los pods de `kube-system` (API server, etcd, scheduler, controller-manager, kube-proxy, CoreDNS).
2. Describe el pod de **etcd** y localiza dónde guarda los datos; comenta por qué su cifrado en reposo y su acceso restringido son críticos.
3. Explora el flujo de autorización: usa `kubectl auth can-i --list` con distintas identidades para ver qué puede hacer cada una.
4. Crea un namespace `app` y despliega un pod; observa el **ServiceAccount** por defecto y su token montado.
5. Comprueba que, por defecto, un pod puede alcanzar a otro en distinto namespace (sin NetworkPolicy).
6. Inspecciona la API del **kubelet** (solo lectura, en laboratorio) y comenta por qué debe requerir autenticación.
7. Dibuja un diagrama del flujo de una petición `kubectl apply`: cliente → API server → authn → authz (RBAC) → admission → etcd → controladores → kubelet.

Ejercicios

1. Enumera cada componente del plano de control y una consecuencia de su compromiso.
2. Explica por qué etcd cifrado en reposo es una defensa clave para los Secrets.
3. Describe la diferencia entre Role y ClusterRole con un ejemplo.
4. Identifica qué escucha en los puertos 6443 y 10250 y su riesgo.
5. Dibuja el flujo authn → authz → admission para una creación de pod.
6. Explica qué aísla un namespace y qué NO aísla por defecto.

Reto verificable

Despliega un clúster de laboratorio y produce un mapa de su superficie de ataque: componentes, puertos que exponen, identidad que usan y qué protege cada control (RBAC, admission, NetworkPolicy).

Criterio de aceptación: el mapa lista API server, etcd, kubelet y scheduler con su puerto y riesgo; identifica el flujo authn→authz→admission; y señala al menos tres controles con el objeto Kubernetes que los implementa.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Pensé que namespace = aislamiento fuerte"	Los namespaces separan nombres, no red ni kernel; añade NetworkPolicy y RBAC.
Token de ServiceAccount montado sin necesidad	Riesgo si el pod se compromete; usa <code>automountServiceAccountToken: false</code> .
etcd accesible sin TLS	Exposición total del estado; exige TLS mutuo y restringe el acceso.
<code>Forbidden</code> al aplicar un manifiesto	RBAC deniega la acción; ajusta el Role/Binding de la identidad.
kubelet API abierta en 10250	Permite exec en pods; exige autenticación y autorización webhook.

Preguntas frecuentes

? ¿Por qué el API server es el objetivo principal de un atacante? Porque es la única puerta a todo el clúster: quien lo controla puede crear pods, leer Secrets y moverse a los nodos. Por eso su autenticación, RBAC y admission control son la defensa central.

? ¿Qué pasa si un atacante lee etcd directamente? Obtiene todo el estado, incluidos los Secrets (por defecto solo en base64, no cifrados). Por eso se recomienda cifrado en reposo de etcd y acceso restringido con TLS mutuo.

? ¿RBAC está activo por defecto? En clústeres modernos sí, pero muchas instalaciones dejan roles amplios o ServiceAccounts con permisos excesivos. RBAC solo protege si se configura con privilegio mínimo.

Referencias

- Martin & Hausenblas, *Hacking Kubernetes*, O'Reilly. <https://www.oreilly.com/library/view/hacking-kubernetes/9781492081722/>
- Kubernetes — Cluster Architecture. <https://kubernetes.io/docs/concepts/architecture/>
- Kubernetes — Controlling Access to the API. <https://kubernetes.io/docs/concepts/security/controlling-access/>
- Kubernetes — RBAC Authorization. <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>
- NIST SP 800-190. <https://csrc.nist.gov/pubs/sp/800/190/final>

Siguiente clase

Clase 229 - Kubernetes: hardening y ataques